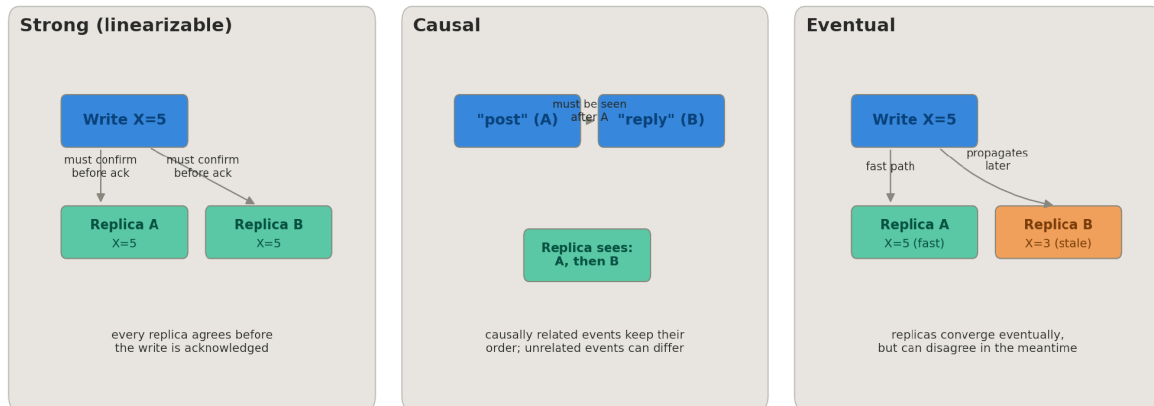


Stage 5 Crash Course: Distributed Systems

This is where the material stops being individually intuitive and starts requiring real precision. Stage 3 gave you multiple copies of data (replication); this stage is about what happens when those copies disagree, or when work needs to be handed off instead of done immediately.

1. Consistency Models

These describe the guarantee a distributed system makes about what a read can return relative to recent writes — a direct refinement of Stage 1's CAP/PACELC framing.

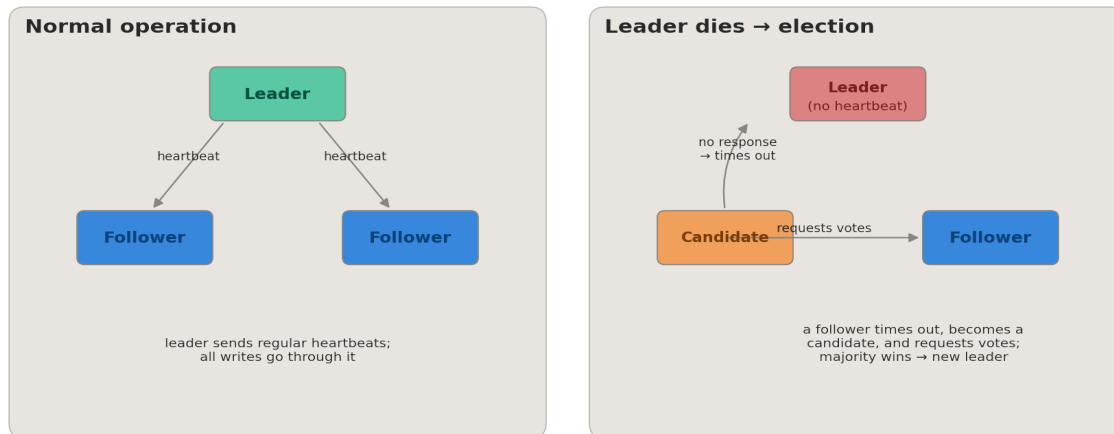


- Strong (linearizable): every read reflects the most recent write, everywhere, immediately — the system behaves as if there were only one copy of the data. Every replica must confirm before the write is acknowledged, which costs latency (PACELC's "L").
- Causal: unrelated events can be seen in different orders by different replicas, but causally related events (a reply can't be seen before the post it replies to) always preserve their order. A middle ground — weaker than strong, more intuitive than eventual.
- Eventual: replicas are allowed to disagree temporarily and converge over time, with no ordering guarantee on how they get there. Cheapest and most available, at the cost of the read being potentially stale or, in some designs, out of order.

Why this matters: "eventually consistent" is often treated as one setting, but it's really a spectrum — the practical question is always "how stale, for how long, and does that staleness matter for this specific feature." A like-count can tolerate it; an account balance generally can't.

2. Consensus (Intuition, Not Proofs)

Consensus is how a group of nodes agrees on one value or one leader despite the possibility of failures and message delays — it's the mechanism underneath leader election, distributed locks, and "exactly one node does this" guarantees.



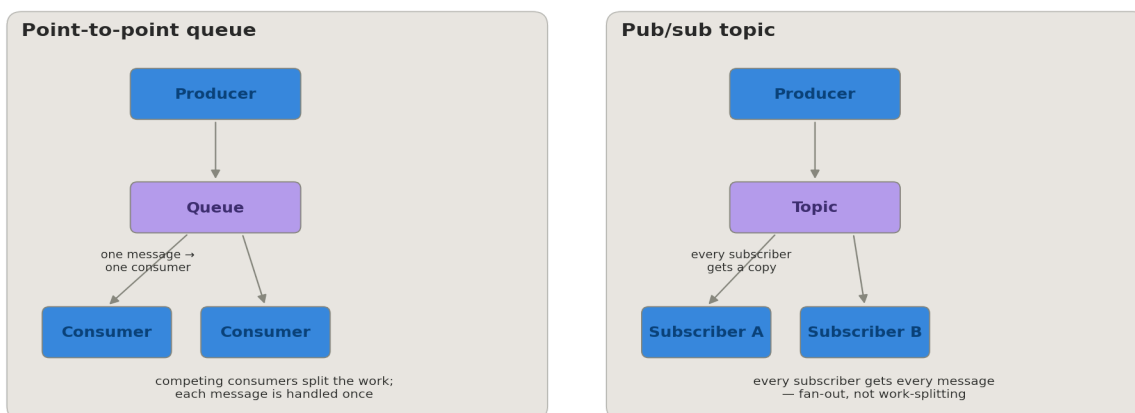
The Raft-style intuition (Paxos solves the same problem with a different, harder-to-follow protocol):

- One node is elected leader and handles all writes; it sends regular heartbeats to prove it's alive.
- If followers stop hearing heartbeats, one becomes a candidate and requests votes from the others.
- Whichever candidate gets a majority of votes becomes the new leader — requiring a majority (not just “some node”) is exactly what prevents two leaders from both believing they're in charge during a network partition.

Why this is hard in practice: networks delay and reorder messages, and nodes can fail silently. Consensus protocols exist specifically to guarantee correctness even under those conditions — that guarantee is expensive (multiple round trips before anything is considered final), which is why consensus is reserved for things that truly need it (leader election, distributed locks, configuration), not for every write in the system.

3. Message Queues

A message queue decouples the producer of work from whoever processes it — the producer doesn't wait for the work to actually finish.



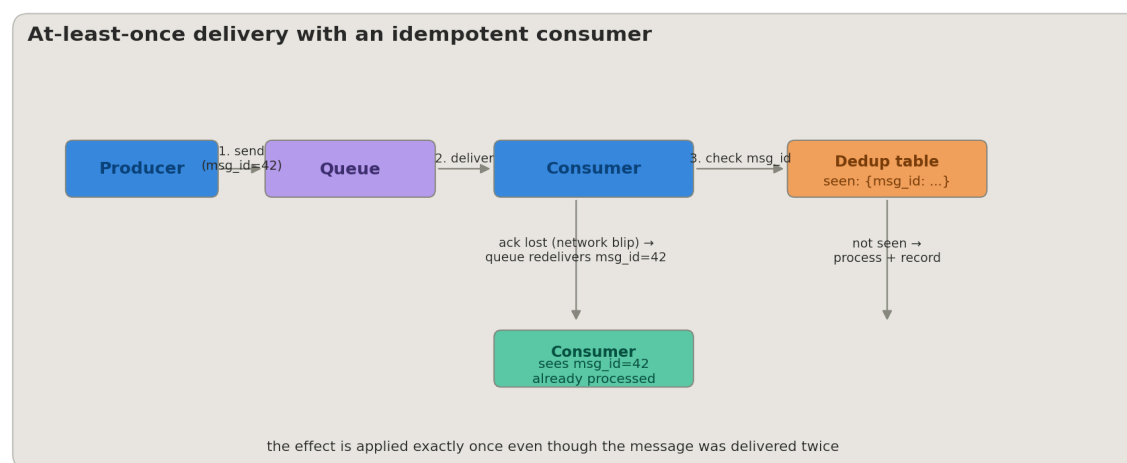
- Point-to-point (queue): each message is delivered to exactly one consumer. Multiple consumers compete for messages, which is how you horizontally scale processing of a single stream of work.
- Pub/sub (topic): every subscriber gets its own copy of every message. This is fan-out, not load-splitting — used when multiple independent systems all need to react to the same event.

	Kafka	RabbitMQ	SQS
--	-------	----------	-----

Model	Log-based; consumers track their own offset	Traditional broker with routing/exchanges	Managed queue, minimal setup
Ordering	Guaranteed within a partition	Guaranteed within a queue (mostly)	Best-effort (standard) or strict (FIFO queues)
Replay	Yes — consumers can re-read old messages	No — once consumed, it's gone	No
Best for	High-throughput event streaming, replay-needed pipelines	Complex routing logic, flexible topologies	Simple, low-ops managed queueing on AWS

4. Idempotency and “Exactly-Once” Delivery

The honest framing: “exactly-once delivery” is effectively a myth at the network layer — a message can always be delivered, lost, or duplicated by a retry after an ack goes missing. What's actually achievable is exactly-once



effect, via an idempotent consumer: the consumer records which message IDs it has already processed, and safely no-ops on a duplicate instead of double-applying it (e.g., double-charging a payment).

- At-least-once delivery (the realistic default): a message may be redelivered if an ack is lost, but is never silently dropped.
- At-most-once delivery: a message is never redelivered, but can be silently lost if the consumer crashes after receiving it but before processing.
- Idempotency key: a unique identifier per logical operation (not per delivery attempt) that lets the consumer recognize “I’ve already done this” regardless of how many times the message arrives.

How this stage connects back

- Consistency models are Stage 1’s CAP/PACELC tradeoff, made precise enough to actually implement against.
- Consensus is the mechanism that makes Stage 3’s “leader–follower replication” concept actually work when the leader dies — it’s the answer to “who decides the new leader, and how do we know they’re right.”
- Message queues are Stage 2’s “async instead of sync” idea, now with the actual machinery (and the actual new failure modes — duplication, ordering, backpressure) that come with it.

Must be able to explain before moving on

1. The difference between causal and eventual consistency, with a concrete example where the difference would actually be visible to a user.

2. Why consensus requires a majority vote specifically, not just any single responding node.
3. When you'd choose Kafka over RabbitMQ, and the reverse.
4. Why “exactly-once delivery” is misleading as a term, and what idempotency actually buys you instead.